PEP 380 – Syntax for Delegating to a Subgenerator

peps.python.org/pep-0380

Author:
Gregory Ewing <greg.ewing at canterbury.ac.nz>

Status:
Final (Accepted and implementation complete, or no longer active)

Created:
13-Feb-2009

Python-Version:
3.3

Resolution:
Python-Dev message

Abstract

A syntax is proposed for a generator to delegate part of its operations to another generator. This allows a section of code containing 'yield' to be factored out and placed in another generator. Additionally, the subgenerator is allowed to return with a value, and the value is made available to the delegating generator.

The new syntax also opens up some opportunities for optimisation when one generator re-yields values produced by another.

PEP Acceptance

Guido officially accepted the PEP on 26th June, 2011.

Motivation

A Python generator is a form of coroutine, but has the limitation that it can only yield to its immediate caller. This means that a piece of code containing a `yield` cannot be factored out and put into a separate function in the same way as other code. Performing such a factoring causes the called function to itself become a generator, and it is necessary to explicitly iterate over this second generator and re-yield any values that it produces.

If yielding of values is the only concern, this can be performed without much difficulty using a loop such as

```
for v in g:
    yield v
```

However, if the subgenerator is to interact properly with the caller in the case of calls to `send()`, `throw()` and `close()`, things become considerably more difficult. As will be seen later, the necessary code is very complicated, and it is tricky to handle all the corner cases correctly.

A new syntax will be proposed to address this issue. In the simplest use cases, it will be equivalent to the above for-loop, but it will also handle the full range of generator behaviour, and allow generator code to be refactored in a simple and straightforward way.

Proposal

The following new expression syntax will be allowed in the body of a generator:

```
yield from <expr>
```

where <expr> is an expression evaluating to an iterable, from which an iterator is extracted. The iterator is run to exhaustion, during which time it yields and receives values directly to or from the caller of the generator containing the `yield from` expression (the "delegating generator").

Furthermore, when the iterator is another generator, the subgenerator is allowed to execute a `return` statement with a value, and that value becomes the value of the `yield from` expression.

The full semantics of the `yield from` expression can be described in terms of the generator protocol as follows:

- Any values that the iterator yields are passed directly to the caller.
- Any values sent to the delegating generator using `send()` are passed directly to the iterator. If the sent value is None, the iterator's `__next__()` method is called. If the sent value is not None, the iterator's `send()` method is called. If the call raises StopIteration, the delegating generator is resumed. Any other exception is propagated to the delegating generator.
- Exceptions other than GeneratorExit thrown into the delegating generator are passed to the `throw()` method of the iterator. If the call raises StopIteration, the delegating generator is resumed. Any other exception is propagated to the delegating generator.
- If a GeneratorExit exception is thrown into the delegating generator, or the `close()` method of the delegating generator is called, then the `close()` method of the iterator is called if it has one. If this call results in an exception, it is propagated to the delegating generator. Otherwise, GeneratorExit is raised in the delegating generator.
- The value of the `yield from` expression is the first argument to the `StopIteration` exception raised by the iterator when it terminates.
- `return expr` in a generator causes `StopIteration(expr)` to be raised upon exit from the generator.

Enhancements to StopIteration

For convenience, the `StopIteration` exception will be given a `value` attribute that holds its first argument, or None if there are no arguments.

Formal Semantics

Python 3 syntax is used in this section.

1. The statement

   ```
   RESULT = yield from EXPR
   ```

   is semantically equivalent to

   ```
   _i = iter(EXPR)
   try:
       _y = next(_i)
   except StopIteration as _e:
       _r = _e.value
   else:
       while 1:
           try:
               _s = yield _y
           except GeneratorExit as _e:
               try:
                   _m = _i.close
               except AttributeError:
                   pass
               else:
                   _m()
               raise _e
           except BaseException as _e:
               _x = sys.exc_info()
               try:
                   _m = _i.throw
               except AttributeError:
                   raise _e
               else:
                   try:
                       _y = _m(*_x)
                   except StopIteration as _e:
                       _r = _e.value
                       break
           else:
               try:
                   if _s is None:
                       _y = next(_i)
                   else:
                       _y = _i.send(_s)
               except StopIteration as _e:
                   _r = _e.value
                   break
   RESULT = _r
   ```

2. In a generator, the statement

   ```
   return value
   ```

   is semantically equivalent to

   ```
   raise StopIteration(value)
   ```

   except that, as currently, the exception cannot be caught by `except` clauses within the returning generator.

3. The StopIteration exception behaves as though defined thusly:

   ```
   class StopIteration(Exception):

       def __init__(self, *args):
           if len(args) > 0:
               self.value = args[0]
           else:
               self.value = None
           Exception.__init__(self, *args)
   ```

Rationale

The Refactoring Principle

The rationale behind most of the semantics presented above stems from the desire to be able to refactor generator code. It should be possible to take a section of code containing one or more `yield` expressions, move it into a separate function (using the usual techniques to deal with references to variables in the surrounding scope, etc.), and call the new function using a `yield from` expression.

The behaviour of the resulting compound generator should be, as far as reasonably practicable, the same as the original unfactored generator in all situations, including calls to `__next__()`, `send()`, `throw()` and `close()`.

The semantics in cases of subiterators other than generators has been chosen as a reasonable generalization of the generator case.

The proposed semantics have the following limitations with regard to refactoring:

- A block of code that catches GeneratorExit without subsequently re-raising it cannot be factored out while retaining exactly the same behaviour.
- Factored code may not behave the same way as unfactored code if a StopIteration exception is thrown into the delegating generator.

With use cases for these being rare to non-existent, it was not considered worth the extra complexity required to support them.

Finalization

There was some debate as to whether explicitly finalizing the delegating generator by calling its `close()` method while it is suspended at a `yield from` should also finalize the subiterator. An argument against doing so is that it would result in premature finalization of the subiterator if references to it exist elsewhere.

Consideration of non-refcounting Python implementations led to the decision that this explicit finalization should be performed, so that explicitly closing a factored generator has the same effect as doing so to an unfactored one in all Python implementations.

The assumption made is that, in the majority of use cases, the subiterator will not be shared. The rare case of a shared subiterator can be accommodated by means of a wrapper that blocks `throw()` and `close()` calls, or by using a means other than `yield from` to call the subiterator.

Generators as Threads

A motivation for generators being able to return values concerns the use of generators to implement lightweight threads. When using generators in that way, it is reasonable to want to spread the computation performed by the lightweight thread over many functions. One would like to be able to call a subgenerator as though it were an ordinary function, passing it parameters and receiving a returned value.

Using the proposed syntax, a statement such as

```
y = f(x)
```

where f is an ordinary function, can be transformed into a delegation call

```
y = yield from g(x)
```

where g is a generator. One can reason about the behaviour of the resulting code by thinking of g as an ordinary function that can be suspended using a `yield` statement.

When using generators as threads in this way, typically one is not interested in the values being passed in or out of the yields. However, there are use cases for this as well, where the thread is seen as a producer or consumer of items. The `yield from` expression allows the logic of the thread to be spread over as many functions as desired, with the production or consumption of items occurring in any subfunction, and the items are automatically routed to or from their ultimate source or destination.

Concerning `throw()` and `close()`, it is reasonable to expect that if an exception is thrown into the thread from outside, it should first be raised in the innermost generator where the thread is suspended, and propagate outwards from there; and that if the thread is terminated from outside by calling `close()`, the chain of active generators should be finalised from the innermost outwards.

Syntax

The particular syntax proposed has been chosen as suggestive of its meaning, while not introducing any new keywords and clearly standing out as being different from a plain `yield`.

Optimisations

Using a specialised syntax opens up possibilities for optimisation when there is a long chain of generators. Such chains can arise, for instance, when recursively traversing a tree structure. The overhead of passing `__next__()` calls and yielded values down and up the chain can cause what ought to be an O(n) operation to become, in the worst case, O(n**2).

A possible strategy is to add a slot to generator objects to hold a generator being delegated to. When a `__next__()` or `send()` call is made on the generator, this slot is checked first, and if it is nonempty, the generator that it references is resumed instead. If it raises StopIteration, the slot is cleared and the main generator is resumed.

This would reduce the delegation overhead to a chain of C function calls involving no Python code execution. A possible enhancement would be to traverse the whole chain of generators in a loop and directly resume the one at the end, although the handling of StopIteration is more complicated then.

Use of StopIteration to return values

There are a variety of ways that the return value from the generator could be passed back. Some alternatives include storing it as an attribute of the generator-iterator object, or returning it as the value of the `close()` call to the subgenerator. However, the proposed mechanism is attractive for a couple of reasons:

- Using a generalization of the StopIteration exception makes it easy for other kinds of iterators to participate in the protocol without having to grow an extra attribute or a close() method.
- It simplifies the implementation, because the point at which the return value from the subgenerator becomes available is the same point at which the exception is raised. Delaying until any later time would require storing the return value somewhere.

Rejected Ideas

Some ideas were discussed but rejected.

Suggestion: There should be some way to prevent the initial call to __next__(), or substitute it with a send() call with a specified value, the intention being to support the use of generators wrapped so that the initial __next__() is performed automatically.

Resolution: Outside the scope of the proposal. Such generators should not be used with `yield from`.

Suggestion: If closing a subiterator raises StopIteration with a value, return that value from the `close()` call to the delegating generator.

The motivation for this feature is so that the end of a stream of values being sent to a generator can be signalled by closing the generator. The generator would catch GeneratorExit, finish its computation and return a result, which would then become the return value of the close() call.

Resolution: This usage of close() and GeneratorExit would be incompatible with their current role as a bail-out and clean-up mechanism. It would require that when closing a delegating generator, after the subgenerator is closed, the delegating generator be resumed instead of re-raising GeneratorExit. But

this is not acceptable, because it would fail to ensure that the delegating generator is finalised properly in the case where close() is being called for cleanup purposes.

Signalling the end of values to a consumer is better addressed by other means, such as sending in a sentinel value or throwing in an exception agreed upon by the producer and consumer. The consumer can then detect the sentinel or exception and respond by finishing its computation and returning normally. Such a scheme behaves correctly in the presence of delegation.

Suggestion: If `close()` is not to return a value, then raise an exception if StopIteration with a non-None value occurs.

Resolution: No clear reason to do so. Ignoring a return value is not considered an error anywhere else in Python.

Criticisms

Under this proposal, the value of a `yield from` expression would be derived in a very different way from that of an ordinary `yield` expression. This suggests that some other syntax not containing the word `yield` might be more appropriate, but no acceptable alternative has so far been proposed. Rejected alternatives include `call`, `delegate` and `gcall`.

It has been suggested that some mechanism other than `return` in the subgenerator should be used to establish the value returned by the `yield from` expression. However, this would interfere with the goal of being able to think of the subgenerator as a suspendable function, since it would not be able to return values in the same way as other functions.

The use of an exception to pass the return value has been criticised as an “abuse of exceptions”, without any concrete justification of this claim. In any case, this is only one suggested implementation; another mechanism could be used without losing any essential features of the proposal.

It has been suggested that a different exception, such as GeneratorReturn, should be used instead of StopIteration to return a value. However, no convincing practical reason for this has been put forward, and the addition of a `value` attribute to StopIteration mitigates any difficulties in extracting a return value from a StopIteration exception that may or may not have one. Also, using a different exception would mean that, unlike ordinary functions, ‘return’ without a value in a generator would not be equivalent to ‘return None’.

Alternative Proposals

Proposals along similar lines have been made before, some using the syntax `yield *` instead of `yield from`. While `yield *` is more concise, it could be argued that it looks too similar to an ordinary `yield` and the difference might be overlooked when reading code.

To the author’s knowledge, previous proposals have focused only on yielding values, and thereby suffered from the criticism that the two-line for-loop they replace is not sufficiently tiresome to write to justify a new syntax. By dealing with the full generator protocol, this proposal provides considerably more benefit.

Additional Material

Some examples of the use of the proposed syntax are available, and also a prototype implementation based on the first optimisation outlined above.

Examples and Implementation

A version of the implementation updated for Python 3.3 is available from tracker issue #11682

Copyright

This document has been placed in the public domain.

Source: https://github.com/python/peps/blob/main/pep-0380.txt

Last modified: 2017-11-11 19:28:55 GMT